

Senior Frontend Engineer Take-Home Assignment: Real-Time Operational Dashboard

Estimated Time Commitment: 8–12 hours over a 2-day window

Stack: React, TypeScript, Modern CSS/Design System, State Management, Cypress/Jest, Docker

1. Context & Business Problem

You are tasked with building a modern frontend prototype for a critical **Real-Time Fleet & Operations Monitoring Dashboard**. Operational managers rely on this dashboard to monitor high-frequency telemetric data (e.g., node health, event streams, system alerts) and make split-second operational decisions.

The existing legacy system suffers from poor UI responsiveness, rendering lag during high-frequency updates, and a lack of clear visual hierarchy. Your goal is to architect a performant, scalable, and intuitive solution using modern React and TypeScript.

2. Technical Requirements & Deliverables

Core Application Capabilities

1. Real-Time Data Feed & Synchronization:

- Integrate or mock a high-frequency real-time data source (WebSocket, Server-Sent Events, or a simulated async stream generator provided in the starter/mock setup).
- Stream metrics such as: **Node ID**, **Status** (OK, WARNING, CRITICAL), **CPU Load (%)**, **Memory Usage (%)**, **Latency (ms)**, and **Timestamp**.
- Ensure state synchronization handles incoming updates smoothly without blocking the main UI thread or causing unnecessary re-renders.

2. Operational Clarity & UI Components:

- **Metrics Overview:** Key high-level counters (Total Active Nodes, Critical Alerts, Average System Latency).
- **Real-Time Grid/List View:** Display active operational nodes. Implement filtering by status and dynamic sorting (e.g., sort by highest latency or CPU load).
- **Data Visualization:** Render a visual representation (e.g., line chart, histogram, or custom metric bars) showing live performance over time for a selected node.
- Design a cohesive UI leveraging modular, reusable components (simulating an in-house Design System).

3. Performance Optimization:

- Optimize rendering for frequent data bursts (e.g., standard React memoization, state colocation, or windowing/virtualization if displaying large data lists).

- Ensure low interface input latency during real-time updates.
- 4. **Testing Strategy:**
 - **Unit/Integration Tests:** Comprehensive unit tests for state reducers/hooks and complex logic using **React Testing Library** or **Jest**.
 - **E2E/Integration Tests:** At least 1–2 end-to-end tests using **Cypress** covering a critical user journey (e.g., filtering nodes by **CRITICAL** status and inspecting telemetry details).
- 5. **Architecture & Documentation (**ARCHITECTURE.md**):**
 - Include a short Markdown document explaining:
 - Architectural decisions and state management strategy.
 - Trade-offs made due to the 2-day time constraint.
 - How you would scale this application for 10,000+ active streaming nodes in production.

3. Containerization Requirements (Mandatory)

The application **must** be fully containerized. Evaluators will clone your repository and run the application inside isolated containers without installing local **node_modules**.

- **Dockerfile:** Multi-stage build for production/development environment.
- **Docker Compose (**docker-compose.yml**):**
 - Configured so running **docker-compose up** builds and starts both the React frontend and any required mock server or assets.
 - Application must be exposed and accessible on **http://localhost:3000** (or **http://localhost:8080**).

4. Anonymity & Submission Protocol (Mandatory Rules)

To ensure an unbiased evaluation process, submissions must adhere strictly to double-blind assessment protocols. **Failure to follow these rules will invalidate the submission.**

Anonymity Constraints

1. **Zero Personal Identifiers:**
 - Do **NOT** include your real name, username, email address, or company affiliation anywhere in the codebase, **package.json**, commit messages, or documentation files.
2. **Anonymous GitHub Repository:**
 - Create a fresh, temporary, public GitHub account or use an anonymous repository link.
 - Clean git history of personal email metadata (e.g., via **git config user.name "Candidate"** and **git config user.email "candidate@anonymous.local"** prior to committing).

3. Candidate Identification Algorithm:

- Generate your unique **Candidate Identifier** using any algorithm or hashing function of your choice (e.g., SHA-256 hash of your favorite quote, a randomly generated UUID v4, or a custom hashing script).
- Include a text file named **CANDIDATE_ID.txt** at the root of your repository containing:
 1. Your generated Candidate Identifier.
 2. A brief 1-line explanation of the algorithm/method used to generate it.

5. Submission Checklist

Before submitting your repository link, ensure the following files exist in the root directory:

Plaintext

None

```
|— Dockerfile
|— docker-compose.yml
|— CANDIDATE_ID.txt
|— ARCHITECTURE.md
|— README.md (Setup instructions, docker commands, test commands)
|— cypress/ (or E2E test directory)
|— src/
```

6. Evaluation Criteria

Submissions will be graded on five key dimensions:

Dimension	Weight	Criteria
Architecture & TypeScript	25%	Clean file structure, strict TypeScript types, effective modular design, clear separation of UI and business logic.



Real-Time & Performance	25%	Efficient state management, prevention of unnecessary render cascades during fast data updates, UI responsiveness.
Containerization & Setup	20%	Flawless <code>docker-compose up</code> execution, proper Docker multi-stage builds, and self-contained environment setup.
Quality & Testing	15%	Robust test coverage (Unit + Cypress E2E), testing meaningful user interactions rather than implementation details.
Operational UI/UX	15%	UX clarity under operational stress, clean modern layout, intuitive visual feedback for critical events.

Instructions for Submission

Send the anonymous public GitHub repository link to your recruiting point of contact before the 48-hour deadline.